

Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barenghi

14/02/2023

Last (family) Name _____
First (given) Name _____
Matricola _____
Codice Persona _____

Professor: ☐ Zanero ☐ Carminati ☐ Barenghi

Have you done any challenges/homework, even partially? ☐ Yes ☐ No

Do you want to withdraw? ☐ Yes ☐ No

Priority Grading ☐ Yes, Motivation:
☐ No

Instructions

- **Duration:** 2.5 hours.
- The exam is composed of 15 pages. Check that you have all of them.
- Just as a cross-check, tell us whether you have completed challenges by appropriately putting an "X" mark.
- The exam is "closed book." The students will not be allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found, and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

- DISTANCE MODE ONLY

- On the computer, you can open only the MS forms browser tab and the exam pdf. Any other application or document is prohibited.
- You will have to share your screen, and leave your microphone opened, and your webcam must frame both you and your workspace.
- You are responsible for having all the necessary technological equipment for the correct exam delivery.
- If you disable the screen sharing or the audio, you will be expelled from the virtual room, and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or a subset at the instructor's decision.
- Regarding the submission:
 - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
 - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
 - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
 - We will evaluate only the uploaded **readable** documents.
 - USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

Consider smart IoT devices are Internet of Things (IoT) devices that are equipped with intelligent features, such as artificial intelligence (AI), machine learning (ML), and advanced connectivity capabilities. They are designed to connect to the internet and other devices, allowing them to exchange data and communicate with each other in real-time. These devices can range from smart home appliances, such as smart thermostats, smart locks, and smart lights, to wearable devices, such as smartwatches and fitness trackers, and even industrial devices, such as smart sensors and robotics. Smart IoT devices often use sensors, actuators, and other hardware components to gather information about their environment and to perform tasks. They can be controlled remotely through a smartphone app, web interface, or other connected devices. Additionally, smart IoT devices often have built-in processing power and storage capacity, enabling them to make decisions and perform actions based on the data they collect and the instructions they receive.

[2 points] 1. What are the main assets at risk in such a scenario? Suggest at least two assets.

Asset	Motivation

[2 points] 2. Suggest, in rough order of importance, the most likely potential threats and threat agents against such infrastructure and their operating companies.

Threats	Threat Agents
Motivation:	Motivation:
Motivation:	Motivation:

Exercise 2 - Introduction to Cryptography (6 points)

[3 points] Consider a data protection mechanism which encrypts an entire hard disk, block by block, by means of AES in Counter (CTR) mode, employing a 128 bit key.

The system administrator, following a new directive which mandates keys to be at least 256 bits long, implements the following compatibility measure: it encrypts the volume again, with the same 128 bit key and counter. Argue on whether the method provides a security margin which is larger, smaller or the same with respect to the original encryption scheme. Describe an alternative measure to comply with the directive, other than decrypting and re-encrypting the entire volume.

The compatibility measure is actually *decrypting* the volume, as applying twice the AES-CTR encryption function with the same key and counter adds via xor the same pseudorandom pad to the Ciphertext. The security margin is clearly lower than before: it's non-existent. Encrypting with AES-CTR and a *different* 128 bit key actually solves the decryption issue, and provides 256 bits of equivalent security (under the largely believed assumption that AES is not a group).

[3 points] Considering the aforementioned scenario, is it possible to claim that the information on the disk cannot be tampered with in a meaningful way, given that all the information on disk is fully encrypted? Either support the claim or disprove it providing a practical example and a solution to prevent tampering.

Encrypting data with AES in counter mode does not provide any protection against tampering. Indeed, an attacker could modify the ciphertext at her own will, knowing that a bit flip in the ciphertext will result in a bit flip in the plaintext, in the same position. Adding a message authentication code (MAC) to the data (e.g., disk-block-wise) prevents tampering altogether.

Exercise 3 - Binary Vulnerabilities (6 points)

```
00 ... #include part ...
01
02 const char *filename = "/tmp/psw_tmp.txt";
03
04 struct User {
05     char username[32];
06     char password[13];};
07
08 void getUsername(struct User *user) {
09     scanf("%s", user->username);}
10
11 void generatePassword(struct User *user) {
12     time_t t;
13     srand((unsigned) time(&t));
14     for (int i = 0; i < 12; i++) {
15         user->password[i] = 'a' + rand() % 26;}
16     user->password[12] = '\0';}
17
18 void writeToFile(const struct User *user) {
19     FILE *fp;
20     fp = fopen(filename, "w");
21     if (fp == NULL) {
22         printf("Error opening file!\n");
23         return;}
24     fprintf(fp, "Username: %s\nPassword: %s\n", user->username,
25 user->password);
26     fclose(fp);}
27
28 void changePermissions(void) {
29     chmod("/tmp/psw_tmp.txt", 0600);
30     chown("/tmp/psw_tmp.txt", 0, 0);}
31
32 int main(void) {
33     struct User user;
34     clearenv(); /* Clean the environment variable */
35     getUsername(&user);
36     generatePassword(&user);
37     writeToFile(&user);
38     changePermissions();
39     printf("Welcome!");
40     printf(user.username);
41     return 0;}
```

- The C standard library is loaded at a known address during every execution of the program, and that the address of the function `system()` is `0xf4d0e2d3`.
- Environment variables are located in the highest addresses.
- *The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdec1** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against exploiting memory corruption errors are present (unless specified otherwise).*

- [1 point]** 1. The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table, focusing on a vulnerability per row.

[2 points] 2. **Focus only on the stack-based buffer overflow(s) you found.** Write an exploit for this vulnerability that **must execute the following shellcode**, composed of 8 bytes, which opens a shell: **0x20 0x30 0x40 0x50 0x60 0x70 0x80 0x90**.

2.a) Describe **all the steps and assumptions** required to successfully exploit the vulnerability. **Include also any assumption on how you must call and run the program**; e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables (if any), the input provided during the execution, if multiple executions are necessary.



2.b) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

[2 points] **3.** Consider ONLY the correctly implemented “Stack Canary” mitigation technique (i.e., randomly generated at each execution). Is it effective in preventing your exploit at point 2.? If the mitigation technique is effective, explain why and describe how you would bypass the mitigation to **execute the same shellcode of the previous question.**

3.a) Describe **all the steps and assumptions** required to successfully exploit the vulnerabilities. Include any assumption on how you must call and run the program: e.g., the values for the command-line arguments required to trigger the exploit correctly and/or environment variables (if any), the input provided during the execution, if multiple executions are necessary, the vulnerability exploited.

3.b) Make sure that you show how the exploit will appear in the process memory with respect to the stack layout right **before and after the execution of the vulnerable line** during the program exploitation showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The content of relevant registers (i.e., EBP, ESP);
- The functions stack frames.

Show also the content of the caller frame.

[1 point] **4.** A friend of yours suggests that there is an alternative way to leak the content of file `"/tmp/psw_tmp.txt"`, while another generates a new password, exploiting a different vulnerability and without executing any shellcode or calling library functions. Please explain **all the steps and assumptions** required for successful exploitation. If it is not possible, please explain why.

Exercise 4 - Web Vulnerabilities (6 points)

PCParts.com is a website that assists users in finding computer components. It offers a comprehensive selection of PC components, allowing you to compare and review them. This makes it the ideal website for anyone seeking to build their own personal computer.

The relevant code of the application is the following:

```
00 app.route("/search_item") # /search_item
01 def search_item(request):
02     if request.method != "GET":
03         return abort(403)
04     if q not in request.args.keys():
05         return render_template("search_item.html")
06     q = request.args["q"]
07     db_query = "SELECT * FROM PRODUCTS WHERE name LIKE '%" + q + "%' OR description
08               LIKE '%" + q + "%';"
09     result = db_execute_query(db_query)
10     if len(result) == 0:
11         return render_template("search_item.html", error= q + " not found!")
12     else:
13         return render_template("search_item.html", products=result)
14
15 app.route("/add_review") # /add_review
16 def add_review(request):
17     if not is_session_valid(session):
18         return redirect(url_for("/login"))
19     if request.method != "POST":
20         return abort(403)
21     user_id = session["user_id"]
22     stars = request.form["stars"]
23     comment = request.form["comment"]
24     prod_id = request.form["prod_id"]
25     db_query = "INSERT INTO REVIEWS VALUES (%d, %d, '%s', %d);"
26     db_execute_query_with_statement(db_query, prod_id, user_id, comment, stars)
27     return redirect(url_for("/products"))
28
29 # search_item.html
30 <html>
31 ...
40 {% if error %}
41     <div class="error">{{ error }}</div>
42 {% endif %}
43 ...
50 {% for product in products %} # Iterating over products
51     <div>
52         <h7{{ product["name"] }}</h7>
53         
54         <p>{{ product["description"] }}</p>
55     </div>
56 {% endfor %}
57 ...
60 </html>
```

The relational database schema used by the website is the following:

Table: **USERS**

user_id (Primary Key) (Integer) (Auto increment)	username (String)	email (String)	password (String)
--	----------------------	-------------------	----------------------

Table: **PRODUCTS**

prod_id (Primary Key) (Integer) (Auto increment)	name (String)	description (String)	img_path (String)
--	------------------	-------------------------	----------------------

Table: **REVIEWS**

prod_id (Primary Key) (Integer) (Referencing PRODUCTS.prod_id)	user_id (Primary Key) (Integer) (Referencing USERS.user_id)	comment (String)	stars (Integer)
--	---	---------------------	--------------------

Assume the following:

- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The function **db_execute_query_with_statement** correctly executes queries with prepared statements.
- The function **render_template** **does not perform any sanitization of the input.**
- The **DBMS** does not support stacked queries (e.g., "SELECT ... ; INSERT INTO ... ;" fails if executed)

More details:

- The function **render_template** generates an html page with optional parameters that are placed in the html code.
- The function **url_for** generates an url of the provided page of the same domain. Optional query parameters of the url can be specified as parameters.
- The function **redirect** redirects the user to the web page of the provided url.
- The function **db_execute_query_with_statement** returns a list of rows as a result, each of which has the fields specified by the **SELECT** statement. For instance, `result[4]["comment"]` is accessing the field "comment" of the 5th row.

1. [3 points] Fill out the following table by answering the questions for each class of vulnerabilities. While answering the questions, be **exhaustive, concise, and straight to the point**. If you believe there might be a vulnerability, specify all the corresponding line/s of code.

<i>Vulnerability class</i>	<i>Is there a vulnerability of this class in the code above? How could an adversary exploit it?</i>	<i>If the vulnerability is present, explain the simplest procedure to remove it.</i>
----------------------------	---	--

SQL Injection (SQL-I)	<i>Line 7. There's a SQL-Injection vulnerability. The attacker may leak data from the DB.</i>	<i>Use prepared statements.</i>
cross-site scripting (XSS) Specify if reflected, stored, DOM...	<i>Error is not sanitized and thus, can be used for a reflected XSS attack.</i>	<i>The simplest procedure would be to apply escaping/filtering to the message.</i>
Cross-site request forgery (CSRF)	<i>Yes, there is no implementation of a CSRF token.</i>	<i>See slides (CSRF tokens, etc.).</i>

[1.5 points] 2. Can you leak the password of the user Law? Explain all the steps you'd take to reproduce the exploit, including the final code you'd write.

To reproduce the exploit, we use the SQL-Injection vulnerability:

q = "some_chars_to_make_the_first_query_return_zero' UNION SELECT 1, password, 1 FROM USERS WHERE username = 'Law'; -- "

[1.5 points] 3. By sending a malicious link of PCParts.com to some friends of yours, can you make an alert appear on her browser that says "Hello Dude!!!" If not, explain why it is not possible. If yes, explain all the steps you'd take to reproduce the exploit, including the final code you'd write

Exploiting the XSS:

q = "<SCRIPT>alert('Hello Dude!!!')</SCRIPT>"

This will work since there will be no item with such name or description. Hence, the same payload will be shown as an error.

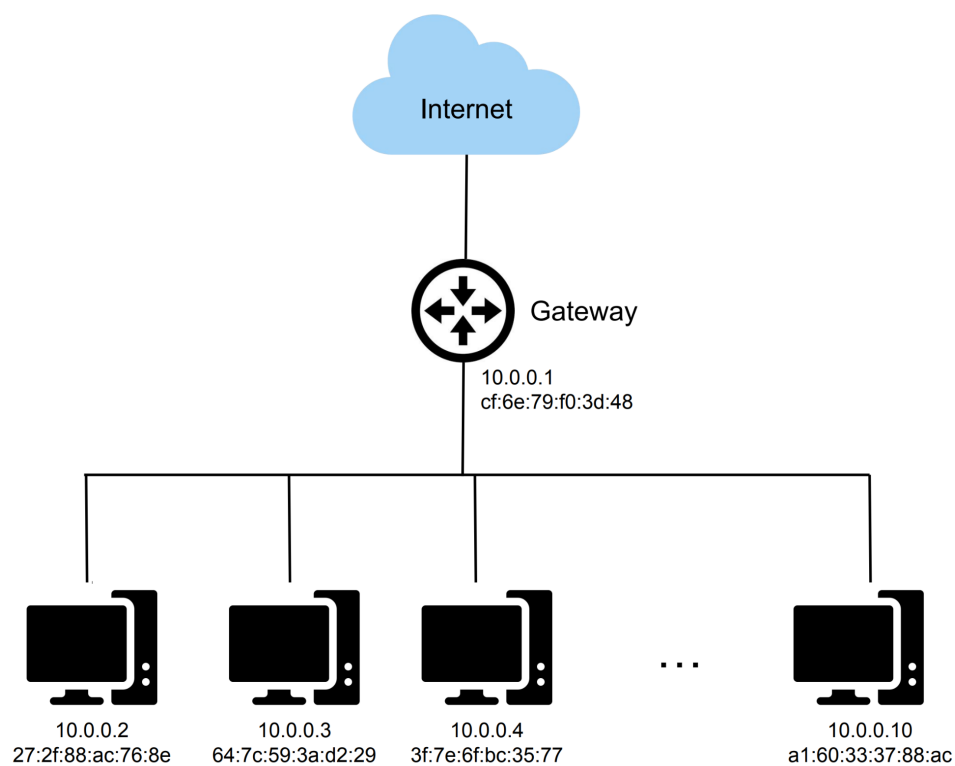
Alternatively, one can mix the SQL-Injection and the XSS vulnerability to make the query return data from the reviews table. Such data is not sanitized, and thus the XSS works as well.

Exercise 5 - Network Security (6 points)

To support those who may not have access to technology at home, the city library has set up some available computers for public use. The structure of the library network is schematized in the figure below.

Unfortunately, some users reported that after they used the computers made available by the library, some of their credentials were stolen. Upon accurate investigation and analysis of the traffic logs, the system administrator identifies a suspicious packet sequence, leading them to conclude that a malicious actor performed a network attack.

Answer the following questions, based on this network topology and on the sequence of packets shown below.



Source IP	Source MAC	Destination IP	Destination MAC	Packet Type
10.0.0.2	27:2f:88:ac:76:8e	171.28.64.2	cf:6e:79:f0:3d:48	ICMP echo request
10.0.0.1	a1:60:33:37:88:ac	10.0.0.2	27:2f:88:ac:76:8e	ICMP Redirect, Gateway: 10.0.0.10
10.0.0.1	cf:6e:79:f0:3d:48	10.0.0.2	27:2f:88:ac:76:8e	ICMP Redirect, Gateway: 10.0.0.1
10.0.0.1	a1:60:33:37:88:ac	10.0.0.2	27:2f:88:ac:76:8e	ICMP Redirect, Gateway: 10.0.0.10
10.0.0.2	27:2f:88:ac:76:8e	171.28.64.2	a1:60:33:37:88:ac	HTTP request

1. [3 points] What kind of attack can be recognized from this sequence of packets? What is the goal of the attacker? What steps are required to perform the attack?

This is an ICMP Redirect attack. The goal of the attacker is (in this specific attack instance) to redirect the traffic of the victim to its machine (MITM). The attacker needs to send an ICMP Redirect message to the victim, spoofing the IP of the gateway, specifying their own IP address as gateway. This way, the default gateway entry for that route will become the attacker's IP.

2. [1 point] What are the IP addresses of the attacker and of the victim?

The ICMP Redirect message indicates 10.0.0.10 as gateway: since the purpose of the attacker is to perform a MITM attack, this IP address either belongs to the attacker's machine or to another machine they control.

The victim is clearly the machine with 10.0.0.2 as IP address, as they are the receiver of the ICMP Redirect message.

3. [2 points] How can the victim detect this attack? Can it be mitigated? If yes, how?

In general, detecting and mitigating ICMP Redirect attacks is non-trivial, as an ICMP Redirect message could be sent by a gateway as response to any of the packets sent by the victim. An easy option to completely avoid ICMP Redirect attacks is to completely disable ICMP Redirect in the OS, trading off performance as it would imply the risk of sending packets over non-optimal routes.

*However, this specific instance of ICMP Redirect attack is easily detectable, as the victim receives several ICMP Redirect messages following a **single** packet sent. In order to mitigate the attack, the victim could either completely disable ICMP Redirect, or apply a more fine-grained control mechanism: if multiple ICMP Redirect messages are received following a single packet sent, drop them both as it is a possible attempt of ICMP Redirect attack.*

Exercise 6 - Malware (6 points)

Consider the following pseudo-code, which describes a code excerpt taken from a malicious program.

```
1. if is_cpu_virtualized():
2.   exit(-1)
3.
4. encryption_key = get_key()
5. payload = decryptor_func(start_address, end_address, encryption_key)
6. goto start_address
...
7. propagate(payload)

a. function propagate(payload):
b.   encryption_key = generate_random_key(128 bytes)
c.   encryption_func = pick_encryption_algorithm()
d.   decryptor_func = get_decryption_func(encryption_func)
e.   decryptor_func = metamorphize_code(decryptor_func)
f.
g.   encrypted_payload = encryption_func(payload, encryption_key)
h.   new_malware = concatenate_code(decryptor_func, encrypted_payload, encryption_key)
i.
j.   infect_devices_in_contact_list(new_malware)
```

6.1 [2 points] Which code-obfuscation techniques does this malware implement? Motivate your answer, indicating all the relevant lines of code (**be concise**). Please note: naming wrong techniques will make you lose points.

Polymorphism.

Before executing, it retrieves the key and decrypts the payload (4, 5).

Before propagating, it selects an enc algorithm and uses it to encrypt the payload (b, c, g). Then, it retrieves the corresponding decryption alg and obfuscates it (d, e). Finally, it composes the new malware by concatenating dec, payload, key (g, h, j)

NOTE: this program is not metamorphic. In order to be metamorphic, the function metamorphize_code should be applied to the payload. Generating a metamorphic decryptor is an essential requirement for the program to be considered polymorphic.

6.2 [2 points] Name any evasive technique implemented by this malware indicating all the relevant lines of code.

Anti-virtualization (1, 2)

6.3 [2 points] Under which assumption could this malware family be detected through signature-based detection or other fully-static techniques? You only need to specify the assumption made, do not describe the detection techniques employed or propose dynamic-analysis-based alternatives.

One could make assumptions on the key generation algorithm not being indistinguishable from uniform random bitstring generation or the encryption algorithms not yielding a ciphertext indistinguishable from a random bitstring. (1 point)

For full marks, the answer that should be given is that the metamorphize_code function leaves several code lines unvaried allowing for signatures/fingerprints to be extracted.